

Note 2: Service Dependency Ordering in Docker Compose

The Problem

When you have multiple services in a Compose file, they often depend on each other. For example, a backend application cannot start successfully if the database is not yet ready to accept connections. Docker Compose needs to know the correct order in which to start services.

The depends_on Field

The depends_on field tells Compose that one service depends on another, and that the dependent service should only be started after the specified service has been started.

Basic Usage

services:

app:

build: ./app

depends_on:

- db

db:

image: postgres:15

environment:

POSTGRES_PASSWORD: secret

In this example, Compose will start db before app. However, there is an important limitation here — depends_on only waits for the container to start, not for the service inside it to be ready. PostgreSQL takes a few seconds to initialise after the container starts, and app may try to connect before the database is ready.

Health Checks and condition in depends_on

To properly wait until a service is truly ready (not just started), Docker Compose supports health check conditions.

Defining a Health Check

services:

db:

image: postgres:15

environment:

POSTGRES_PASSWORD: secret

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 10s

timeout: 5s

retries: 5

start_period: 10s

- test — the command to run to check health; exit code 0 means healthy
- interval — how often to run the check
- timeout — how long to wait for a response
- retries — how many consecutive failures before marking as unhealthy
- start_period — grace period at startup before failures count

Using condition in depends_on

services:

app:

build: ./app

depends_on:

db:

condition: service_healthy

db:

image: postgres:15

environment:

POSTGRES_PASSWORD: secret

healthcheck:

```
test: ["CMD-SHELL", "pg_isready -U postgres"]
```

```
interval: 10s
```

```
timeout: 5s
```

```
retries: 5
```

Now app will not start until db passes its health check. The three available conditions are:

Condition	Meaning
-----------	---------

service_started	Default — waits for container to start
-----------------	--

service_healthy	Waits for health check to pass
-----------------	--------------------------------

service_completed_successfully	Waits for a one-off container to exit with code 0
--------------------------------	---

Service Restart Policies

In production-like setups, services should restart automatically if they crash.

services:

```
app:
```

```
  image: myapp:latest
```

```
  restart: always
```

Restart Policy	Behaviour
----------------	-----------

no	Never restart (default)
----	-------------------------

always	Always restart regardless of exit code
--------	--

on-failure	Restart only if container exits with non-zero code
------------	--

unless-stopped	Always restart unless manually stopped
----------------	--

Controlling Startup and Shutdown Order

- Compose starts services in dependency order based on `depends_on`
 - Compose stops services in reverse dependency order — dependents are stopped before the services they depend on
 - If no `depends_on` is specified, Compose starts all services in parallel
-